CSC 3320: System-Level Programming

Chapter 2: C Fundamentals

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Announcement

- HW2 will be posted today (Deadline February 10, 11:59 pm)
- TAs are grading HW1, feedback will be revealed soon.

Functions

A ***function*** is a series of statements that have been grouped together and given a name. **Similar to method in Java**

Library functions are provided as part of the C implementation.

A function that computes a value uses a `return` statement to specify what value it "returns":

```
return x + 1;
```

The main Function

The `main` function is mandatory.

`main` is special: it gets called automatically when the program is executed.

`main` returns a status code; the value 0 indicates normal program termination.

If there's no `return` statement at the end of the `main` function, many compilers will produce a warning message.

Statements

A ***statement*** is a command to be executed when the program runs.

`pun.c` uses only two kinds of statements. One is the `return` statement; the other is the ***function call.***

Asking a function to perform its assigned task is known as ***calling*** the function.

C requires that each statement end with a semicolon, except the compound statement

Printing Strings

When the `printf` function displays a ***string literal***—characters enclosed in double quotation marks—it doesn't show the quotation marks.

`printf` doesn't automatically advance to the next output line when it finishes printing.

To make `printf` advance one line, include `\n` (the ***new-line character***) in the string to be printed.

What will be covered?

Identifiers

Keywords

Variable and assignments **Overlaps Chapter 7**

Identifiers

Names for variables, functions, macros, and other entities are called ***identifiers***

An identifier may contain letters, digits, and underscores, but must begin with a letter or underscore.

- E.g.
  - `times10` `get_next_char` `_done` **legal**
  - `10times` `get-next-char` **illegal**

C is ***case-sensitive***

- E.g. the following identifiers are all different
  - `job` `joB` `jOb` `jOB` `Job` `JoB` `JOb` `JOB`

Keywords

The following ***keywords*** can't be used as identifiers:

auto	enum	restrict*	unsigned
break	extern	return	void
case	float	short	volatile
char	for	signed	while
const	goto	sizeof	_Bool*
continue	if	static	_Complex*
default	inline*	struct	_Imaginary*
do	int	switch	
double	long	typedef	
else	register	union	

*C99 only

Variable and assignments

Basic Data types
Constants
Reading and Writing
Naming a variable
Declaring a variable
Assignment
Initialization
Type Conversion
User Defined Types

Basic Data Types

Type	Bytes	Range	Bytes
char	1	-128 … 127	1
short	2	-32768…32767	2
int	4	-2,147,483,648 to 2,147,483,647	4
long	4	-2,147,483,648 to 2,147,483,647	8
long long	8	-2^{63}… $+2^{63}-1$	8
float	4	3.4E+/-38 (7 digits)	4
double	8	1.7E+/-308 (15 digits)	8

For 32-bit machine **For 64-bit**

Tips:

- *How to check if your Unix system is running 32 or 64 bit mode via Terminal?*

Answer: **$uname –m**

- *How to check the exact size of a type or a variable in C on your computer?*

Answer: You can use the sizeof operator. e.g. **sizeof(int) , sizeof(x)**

An Example of sizeof Operator

checkTSize.c

```
#include<stdio.h>

int main(){
    printf("Type char in C has %d bytes\n", sizeof(char));
    printf("Type short in C has %d bytes\n", sizeof(short));
    printf("Type int in C has %d bytes\n", sizeof(int));
    printf("Type long in C has %d bytes\n", sizeof(long));
    printf("Type long long in C has %d bytes\n", sizeof(long long));
    printf("Type float in C has %d bytes\n", sizeof(float));
    printf("Type double in C has %d bytes\n", sizeof(double));
}
```

```
$gcc -o checkTSize checkTsize.c
$./checkTSize
Type char in C has 1 bytes
Type short in C has 2 bytes
Type int in C has 4 bytes
Type long in C has 8 bytes
Type long long in C has 8 bytes
Type float in C has 4 bytes
Type double in C has 8 bytes
```

Basic Data Types

Integers can be unsigned or signed

Type	Bytes 32-bit machine	Range
char	1	-128 … 127
unsigned char	**1**	**0 … 255**
short	2	-32768…32767
unsigned short	**2**	**0 to 65,535**
int	4	-2,147,483,648 to 2,147,483,647
unsigned int	**4**	**0 to 4,294,967,295**
long	4	-2,147,483,648 to 2,147,483,647
unsigned long	**4**	**0 to 4,294,967,295**